

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

COURSE CODE: CSA5112

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS: Total Marks: 50

Annexure A

Code of Conduct:

I _E. Lakshmi Narayanan(192524249) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

E. Lakshmi Narayanan

EAnnexure A

Debugging & Optimisation Task

1. Debugging Hash Function Implementation (SHA Family)

A Python program implementing a SHA-based hashing algorithm produces inconsistent hash outputs for the same input.

Identify errors in message padding and block processing steps.

Fix issues related to bitwise operations and endianness handling.

Optimize the implementation to improve processing speed for large files (≥ 50 MB).

Evaluate how secure hash functions improve integrity compared to basic checksum methods.

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation

A Python-based simulation of SSL/TLS handshake shows delays and occasional handshake failures.

Debug issues in certificate validation and key exchange steps.

Optimize the handshake process to reduce latency.

Refactor the code to reuse session keys efficiently (session resumption).

Analyze the performance vs security trade-offs in TLS optimization.

3. Debugging and Optimizing Digital Signature Algorithm (DSA)

A Python implementation of DSA generates invalid signatures during verification.

Identify issues in parameter selection and random number generation.

Fix errors in signature generation and verification steps.

Optimize modular arithmetic operations for faster execution.

Evaluate the impact of randomness quality on signature security.

4. Debugging Key Management System

A Python-based key management system fails to securely store and retrieve cryptographic keys.

Identify flaws in key storage (plaintext storage, improper encryption).

Fix issues related to key lifecycle management (generation, distribution, rotation).

Optimize secure storage using encryption and access control mechanisms.

Analyze how poor key management can compromise an entire cryptographic system.

5. Optimization of Secure File Transfer Protocol (SFTP-like System)

A Python script simulating secure file transfer experiences slow transfer speeds and occasional data corruption. Debug issues in encryption/decryption synchronization.

Fix errors in packet handling and integrity verification. Optimize throughput for large file transfers (≥ 100 MB). Compare the optimized

1. Debugging Hash Function Implementation (SHA Family)

Errors Identified

a) Message Padding

- Padding bits were added incorrectly.
- Original message length was not appended as a 64-bit value.
- Message was not padded to a multiple of 512 bits.

b) Block Processing

- Message blocks were processed with incorrect sizes.
- Final partial block was ignored.
- Compression function skipped some rounds.

c) Bitwise Operations

- Incorrect use of left shift ($\ll$) instead of rotate operations.
- Missing 32-bit masking ($\& 0xFFFFFFFF$) causing integer overflow issues.

d) Endianness

- SHA uses **big-endian** format.
- Data was interpreted in little-endian order, producing incorrect hashes.

Fixes

- Implement correct SHA padding.
- Process every 512-bit block.
- Replace shifts with rotate operations.
- Use big-endian conversion while reading message words.
- Apply 32-bit masking after every operation.

Optimization

- Read files in chunks (e.g., 4 MB).
 - Avoid loading entire files into memory.
 - Use Python's built-in hashlib library.
-
- Reduce unnecessary copying of byte arrays.

Security Evaluation

SHA Hash	Basic Checksum
Cryptographically secure	Not secure
Collision resistant	Easy to forge
Detects malicious changes	Detects only accidental errors
Used in SSL, Digital Signatures	Used mainly for error detection

Conclusion: Secure hash functions provide strong integrity verification and protect against intentional tampering, unlike simple checksum methods.

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation

Issues Found

Certificate Validation

- Expired certificates accepted.
- Certificate chain not verified.
- Hostname verification missing.
-

Key Exchange

- Public keys exchanged incorrectly.
- Shared secret generated differently on client and server.
- Handshake timeout caused failures.

Fixes

- Verify certificate validity period.
- Validate certificate chain.
- Match hostname with certificate.
- Correct Diffie-Hellman/ECDHE key exchange implementation.

Optimization

- Enable **Session Resumption**.
- Cache session tickets.
- Reduce unnecessary certificate verification.
- Reuse established session keys.

Session Resumption

Instead of performing the complete handshake every time:

1. Store session ID.
2. Reuse previously negotiated symmetric keys.
3. Skip expensive public-key operations.

Result:

- Lower latency
- Faster reconnection
- Reduced CPU usage

Performance vs Security

Optimization	Performance	Security
Session Resumption	High	Slight reduction if session keys remain valid too long
ECDHE	High	Excellent Forward Secrecy
AES-GCM	Fast	Very Secure
Certificate Caching	Faster	Safe with proper validation

Conclusion: TLS optimization improves speed while maintaining strong security when session management is implemented correctly.

3. Debugging and Optimizing Digital Signature Algorithm (DSA)

Problems Identified

Parameter Selection

- Invalid prime numbers (p, q)
- Incorrect generator (g)

Random Number Generation

- Reused random nonce (k)
- Weak random number generator

This causes invalid signatures and private key leakage.

Fixes

- Generate valid DSA parameters.
- Use cryptographically secure random numbers (secrets module).
- Ensure every signature uses a unique nonce.
- Correct signature verification formula.

Optimization

- Use Python's built-in modular exponentiation:

`pow(base, exponent, modulus)`

instead of repeated multiplication.

- Cache repeated calculations.
- Reduce unnecessary modular reductions.

Randomness Impact

Good Randomness	Poor Randomness
------------------------	------------------------

Secure signatures	Private key leakage
-------------------	---------------------

Unique signatures	Signature forgery
-------------------	-------------------

Strong authentication	Easy attacks
-----------------------	--------------

Conclusion: High-quality randomness is essential for DSA security because nonce reuse can expose the private key.

4. Debugging Key Management System

Problems Identified

Insecure Storage

- Keys stored in plaintext.
- Keys hardcoded in source code.
- Weak file permissions.

Lifecycle Issues

- No key rotation.
- No expiration.
- No revocation mechanism.
- Poor key distribution.

Fixes

- Encrypt stored keys using AES.
- Protect master keys.
- Implement automatic key rotation.
- Secure key distribution using TLS.

Optimization

- Encrypt key database.
- Restrict access using authentication.
- Apply role-based access control (RBAC).
- Log every key access.

Impact of Poor Key Management

Good Key Management	Poor Key Management
---------------------	---------------------

Confidentiality maintained	Data exposure
----------------------------	---------------

Secure communication	Key theft
----------------------	-----------

Controlled access	Unauthorized access
-------------------	---------------------

Easy recovery	Complete system compromise
---------------	----------------------------

Conclusion: Even strong encryption becomes ineffective if cryptographic keys are poorly managed.

5. Optimization of Secure File Transfer Protocol (SFTP-like System)

Problems Identified

Encryption Synchronization

- Sender and receiver used different encryption states.
- IV (Initialization Vector) mismatch.

Packet Handling

- Packets received out of order.
- Missing sequence numbers.
- Integrity checks not verified.

Data Corruption

- Missing authentication tag verification.
- Corrupted packets accepted.

Fixes

- Synchronize encryption keys and IV.
- Add packet sequence numbers.
- Verify HMAC or SHA-256 hash for every packet.
- Retransmit corrupted packets.

Optimization

- Increase packet size.
- Use buffered I/O.
- Compress files before encryption.
- Use multithreading for encryption and transfer.
- Transfer files in chunks (e.g., 4–8 MB).

Comparison

Feature	Insecure File Transfer	Optimized SFTP
Encryption	No	Yes
Authentication	No	Yes
Integrity Check	No	SHA-256/HMAC
Confidentiality	No	Strong
Large File Performance	Moderate	High (optimized)
Data Corruption Protection	No	Yes

Conclusion: The optimized SFTP system ensures confidentiality, integrity, and authentication while improving transfer speed for large files through chunked transfers, buffering, and efficient encryption.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation